Teknik Değerlendirme Raporu — NL2SQL Agent (PoC)

Hazırlayanlar: Elif Beyza Güneş, Pınar Atabey

Referans Tarihi: 2026-06-30

Geliştirme Ortamı: macOS / Python 3.12 / SQLite

1. NEDEN BU MİMARAYI SEÇTİK?

Projede doğal dille sorulan iş sorularını SQL'e dönüştürürken hem sistemin kontrolünün tamamen bizde olması hem de güvenlikten ödün vermemek ana hedefimizdi. Sistemimiz 57/57 Pytest birim/geliştirme testinden ve 16/16 canlı kabul ve güvenlik testinden %100 başarıyla geçerek doğrulanmıştır.

LangChain/LlamaIndex Yerine Kendi REST İstemcimiz (llm.py)

İlk başta hazır framework'ler kullanmayı düşündük fakat LangChain gibi yapılar arka planda ne yaptığını gizlediği için bir hata çıktığında müdahale etmek zorlaşıyordu. Bu yüzden doğrudan kendi REST istemcimizi (llm.py) yazdık. Böylece modele giden sistem talimatlarını (system_instruction), parametreleri ve dönen yanıtları milisaniye seviyesinde kontrol edebildik. Hem projenin bağımlılıklarını azalttık hem de karmaşanın önüne geçtik.

Çift Katmanlı Güvenlik Yaklaşımımız (Defense-in-Depth)

Dil modellerinin bazen beklenmedik çıktılar üretebildiğini bildiğimiz için, güvenliği asla sadece LLM'in promptuna bırakmadık. Kod tarafında çift katmanlı bir zırh kurduk:

- **Kod Seviyesi (AST Analizi - sqlglot):** Modelden gelen SQL metnini veritabanına göndermeden önce sql_security.py içinde sqlglot kütüphanesiyle mantıksal ağacına (AST) ayırdık. Yorum satırlarını (/* ... */) tamamen temizledik. Sadece tek bir SELECT çalışmasına izin verdik; DELETE, DROP, UPDATE gibi veritabanını bozacak komutları ve sqlite_master gibi sistem tablolarına erişimleri daha veritabanına ulaşmadan kodda engelledik.
- **Veritabanı Seviyesi (SQLite Read-Only URI & Native Authorizer):** Kod katmanını bir şekilde geçen bir şey olursa diye veritabanı bağlantımızı mode=ro (salt-okunur) açtık. Ayrıca SQLite'ın kendi C seviyesindeki set_authorizer kancasını aktif ederek veritabanı motorunun yetkisiz bir işlem yapmasını kökten engelledik.

Model Seçimi (gemini-3.5-flash-lite)

İlk testlerimizde daha büyük modeller kullandığımızda anlık API yoğunluklarından dolayı 60 saniyelik zaman aşımlarına takılıyorduk. Bu yüzden daha hızlı ve kararlı çalışan gemini-3.5-flash-lite modeline geçtik. Çıkarım hızını maksimuma çıkarmak için de thinkingLevel=LOW ayarını seçtik.

2. KARŞILAŞTIĞIMIZ SORUNLAR VE NASIL ÇÖZDÜK?

- **Göreceli Tarihler ("Geçen ay", "Son 30 gün"):** Testleri farklı günlerde çalıştırdığımızda sistem saati değiştiği için sonuçlar tutarsız çıkıyordu. Çözüm olarak `.env` dosyasına sabit bir referans tarihi (`2026-06-30`) ekledik ve bunu modelin sistem promptuna verdik. Artık model tüm göreceli tarihleri bu sabit gün üzerinden doğru hesaplıyor.
- **Yorum Satırı Arkasına Gizlenen Enjeksiyonlar:** `SELECT * FROM Customers /* DELETE FROM Customers */` gibi zararlı ifadeleri yakalamak için düz metin (regex) araması yetersiz kalıyordu. `sqlglot` parser'ı kullanarak yorum satırlarını tamamen temizledik ve sadece temiz SQL gövdesini değerlendirmeye aldık.
- **Canlı API Zaman Aşımları:** Ücretsiz API katmanında anlık takılmalar yaşandığı için `llm.py` içerisine otomatik yeniden deneme (retry) mekanizması koyduk ve zaman aşımı süresini 60 saniyeye çıkardık.

3. SİSTEMİMİZİN SINIRLILIKLARI

1. **API Yanıt Süreleri:** Yanıt sürelerimiz doğrudan Google API'sinin anlık durumuna bağlı. Bazen 10 saniyede yanıt alırken bazen 40-50 saniyelere çıkabiliyor.
2. **Veritabanında Olmayan Soyut Kavramlar:** Veritabanında karşılığı olmayan "en iyi müşteri" veya "sadık kullanıcı" gibi net olmayan isteklerde sistem ezbere SQL üretmek yerine kullanıcıdan netleştirme (`clarification`) istiyor.
3. **Sentetik Veri Boyutu:** Testlerimizi deterministik olarak üretilen 300 müşteri, 60 ürün, 2.400 sipariş ve 8.395 sipariş kaleminden oluşan FMCG veri setiyle yaptık. Milyonlarca satırlı dev veritabanlarında performansın nasıl etkileneceğini üretim ortamında indekslerle optimize etmek gerekecektir.

4. GELECEKTE NELER EKLEYEBİLİRİZ? (İYİLEŞTİRME ÖNERİLERİ)

- **Semantik Sözlük (Semantic Layer):** "Sadık müşteri" gibi iş terimlerini veritabanı şemasıyla otomatik eşleştiren bir metrik sözlüğü ekleyebiliriz.
- **RAG ile Şema Seçimi:** İleride yüzlerce tablosu olan bir veritabanına geçersek, soruyla ilgili tabloları bir Vektör Veritabanı (Vector DB) üzerinden bulup prompta sadece o tabloları verebiliriz.
- **Önbellekleme (Caching):** Sık sorulan soruların SQL karşılıklarını Redis'te saklayarak API maliyetini azaltıp yanıt süresini milisaniyelere düşürebiliriz.

5. KULLANILAN YAPAY ZEKA ARAÇLARI

Projenin geliştirme sürecinde mimari kurgu, pytest ve kabul testlerinin otomasyonu, güvenlik denetimlerinin yazımı ve dokümantasyon aşamalarında OpenAI Codex ve Gemini asistanlarından faydalanılmıştır.